# ReferralOS - Test Guide

### *What still needs testing, and exactly how to do it*

Version for the build of 27 Oct 2026 (commit a9f6ef5). Windows commands; paths assume the project is at `C:\Users\anime\Desktop\Referral Portal`.

## Why this guide exists

The automated tests prove the logic and the safety rules, but only with **fictional data** and **simulated** Gmail/AI. They cannot tell you whether your real resume is read correctly, whether your real LinkedIn export imports cleanly, whether the ranking makes sense for your network, or whether a real Gmail account sends. Those are the tests in Part C, and only you can run them, because they use your own data.

| Area | Already verified | You still need to verify |
|---|---|---|
| Backend logic (402 tests) | Yes, SQLite + PostgreSQL | Re-run after any change (Part B) |
| Browser flow (Playwright) | Yes, with fake data | Re-run after any change (Part B) |
| Resume reading | Synthetic resumes only | Your real resume (C1) |
| GitHub / portfolio | GitHub API once, read-only | Your account and site (C2) |
| CSV imports | Synthetic 11k rows | Your real LinkedIn export (C3) |
| Contact ranking | Rules tested | Does the order make sense to you (C4) |
| Job boards | 5 public board APIs, once | Companies you care about (C5) |
| Resume tailoring | Rules tested | Read the real PDF line by line (C6) |
| Outreach drafts | Rules tested | Read as the recipient (C7) |
| Gmail sending | Simulated only - never real | Dry run, then self-test, then one real (C8) |
| Privacy / deletion | Tested | Export, purge, delete (C9) |
| docker compose | Not run | Optional (Part D) |

**Safety rule for the whole guide:** nothing you do in C1-C7 can send a message. Keep **Delivery mode = Manual** (or Dry run) until test C8. Do not connect real Gmail before C1-C7 look right.

## How to use it

- Do Part A once (start the app).
- Do Part B whenever code changes (5-10 minutes, automatic).
- Do Part C in order. Each test lists: goal, what to have ready, numbered steps, **Pass if**, and what to write down.
- Fill in the results sheet at the end (Part E) and send it back with screenshots of anything wrong (blur personal details).

## Part A - Start the app (once, about 10 minutes)

**You need:** Python 3.11 with `uv`, Node.js 20 or newer, and three terminal windows. No `.env` file is needed for local testing: the defaults are SQLite and offline (mock) mode.

**Important:** do **not** copy `.env.example` into `backend\`. That file is for docker compose and points the database at a host called `db`, which does not exist outside Docker.

### *Terminal 1 - the API*

```
cd "C:\Users\anime\Desktop\Referral Portal\backend"
uv sync
uv run alembic upgrade head
uv run uvicorn app.main:app --port 8000
```

Wait for `Application startup complete`. Check: open http://localhost:8000/docs - the API page loads.

### *Terminal 2 - the background worker*

```
cd "C:\Users\anime\Desktop\Referral Portal\backend"
uv run python -m app.worker
```

Leave it running. Imports, parsing, scoring, matching and sending happen here. If it is not running, those jobs stay "queued" forever.

### *Terminal 3 - the web app*

```
cd "C:\Users\anime\Desktop\Referral Portal\frontend"
npm install
npm run dev
```

Open **http://localhost:3000**.

### *Create your account*

1. Click **Sign up**, enter your email and a password (at least 10 characters, with a letter and a digit).
2. No real email is sent in local mode. Open the newest file in `backend\data\outbox\`, copy the verification link from it, and open it in the browser.
3. Sign in. The onboarding wizard opens; you can follow it or skip and use the menu.

**Pass if:** you reach the Dashboard signed in. **If not:** see Troubleshooting (Part F).

### *Offline vs real mode*

Steps C1, C3, C4, C6, C7 work fully offline. C2 (GitHub/portfolio) and C5 (job boards) need internet: stop Terminal 1 and 2 (Ctrl+C) and restart them with real mode:

```
$env:ADAPTER_MODE="real"; uv run uvicorn app.main:app --port 8000      # Terminal 1 (PowerShell)
$env:ADAPTER_MODE="real"; uv run python -m app.worker                 # Terminal 2 (PowerShell)
```

Real mode only makes read-only requests to public sites and APIs. It still cannot send email without Gmail connected and your approval.

## Part B - Automated tests (run after every code change)

These need no data from you. Close the app first (Ctrl+C in all three terminals) so ports 3100/8100 are free.

### *B1. Backend tests on SQLite (about 4 minutes)*

```
cd "C:\Users\anime\Desktop\Referral Portal\backend"
uv run pytest -q -p no:warnings
```

**Pass if:** the last line says `402 passed` (or more) and **0 failed**.

### *B2. Browser tests (about 2 minutes)*

```
cd "C:\Users\anime\Desktop\Referral Portal\frontend"
npx playwright test
```

Starts its own throwaway API (port 8100) and web app (port 3100) with fake data, then clicks through sign-up to account deletion, checks 20 pages at phone width, and runs an accessibility audit. **Pass if:** `3 passed`. To watch it: `npx playwright test --headed`. On failure: `npx playwright show-report`.

### *B3. Backend tests on PostgreSQL (optional, about 12 minutes)*

Needed only before running with several users. Requires Docker Desktop to be running.

```
docker run -d --name referralos-test-pg -e POSTGRES_USER=referralos -e POSTGRES_PASSWORD=testpw ^
  -e POSTGRES_DB=referralos_test -p 55432:5432 postgres:16
cd "C:\Users\anime\Desktop\Referral Portal\backend"
$env:TEST_DATABASE_URL="postgresql+psycopg://referralos:testpw@localhost:55432/referralos_test"
uv run pytest -q -p no:warnings
Remove-Item Env:TEST_DATABASE_URL; docker rm -f referralos-test-pg
```

(In PowerShell, put the `docker run` command on one line instead of using ^.)

**Pass if:** 0 failed. Postgres catches things SQLite ignores (text length limits, NUL bytes) - it found two real bugs last time.

### *B4. One test only (when something fails)*

```
uv run pytest -q -p no:warnings tests/test_outreach_flow.py -k linkedin
```

`docs/ACCEPTANCE.md` lists which test proves each of the 14 required scenarios.

## Part C - Real-data tests (you run these)

Do them in order: later tests use what earlier ones created. Allow 2-3 hours in total. Keep a notes file open for the "Write down" items.

### *C1 Resume -> profile accuracy*

**Goal:** The app reads your real resume correctly and never invents anything.

**Have ready:** Your resume as PDF; the DOCX version if you have one; a two-column version if you have one. Offline mode is fine.

**Steps:**

1. Menu **Profile** -> upload your PDF. Wait until its status is **parsed** (the worker must be running).
2. Click **Compare extracted data with your original resume**. The original text is shown next to what was extracted.
3. Go section by section (experience, education, projects, skills). For each entry check: is it there, is it correct, does it point to the right line of the original?
4. Look specifically for: two jobs merged into one, one job split in two, dates wrong (especially *Present*), title and company swapped, skills in the wrong group.
5. Menu **Diagnostics** -> Resume parsing: note the parser, columns detected, items extracted, mean confidence, items dropped, warnings.
6. Edit one wrong entry and save. Then open **Profile -> versions**.
7. Upload the DOCX and the two-column PDF too, and repeat steps 2-4 for each.
8. Optional: upload `samples\resumes\scanned_image_only.pdf`.

**Pass if:**

- Every entry shown can be traced to a line in your original - nothing appears that is not in your resume.
- Conflicts and duplicate roles are listed for you to resolve, never merged silently.
- The edited entry is marked *confirmed by you* and a new profile version exists.
- The scanned file is reported as image-only (there is no OCR) instead of guessed.

**Write down:** Number of entries in your resume, number that needed correcting (= error rate), and each wrong one with what it should be.

### *C2 GitHub and portfolio*

**Goal:** Public repos become suggestions only; nothing is trusted until you confirm it.

**Have ready:** Real mode (Part A). Your GitHub username; your portfolio URL if you have one.

**Steps:**

1. **Settings** (or Profile) -> allow GitHub, enter your username, start the import.
2. Open the suggested projects in **Profile**.
3. Allow portfolio, enter one URL, start the import.
4. Optional: try a site whose robots.txt disallows crawlers.

**Pass if:**

- Repos appear as *suggestions*, not confirmed facts.
- Forks and tutorial/template repos are flagged *possibly not original*.
- Languages become *used in project* skill candidates; nothing becomes confirmed until you confirm it.
- Private repos never appear. A disallowed site is refused with an explanation.

**Write down:** Any repo flagged wrongly or not flagged when it should be.

### *C3 Imports and data quality*

**Goal:** Your real LinkedIn export imports without silently losing anyone.

**Have ready:** LinkedIn *Connections.csv*: LinkedIn -> Settings & Privacy -> Data privacy -> Get a copy of your data -> choose *Connections* -> wait for the email (can take up to a day) -> unzip. Optionally an HR/recruiter list you hold lawfully.

**Steps:**

1. Menu **Imports** -> LinkedIn -> upload *Connections.csv*.
2. Check the detected encoding, delimiter and header line, the suggested column mapping and the first rows. Fix the mapping if needed, then start.
3. When done, read the report and check: **rows read = valid + duplicate + invalid**.
4. Click **Inspect rows** for invalid and duplicate rows. Are the reasons right? Are two different people with the same name wrongly called duplicates?
5. Open **Companies** -> the match review queue. Do "Acme Inc." and "Acme" collapse sensibly? Ambiguous ones must wait for you.
6. Optional HR list: import it. It must ask where the list came from. Choose *self-collected* (a purchased/scraped list blocks bulk sending).
7. Add one address to **Do-not-contact**, import the same file again, and check that address stays suppressed.
8. Check **Diagnostics -> Imports** and the **Dashboard** percentages (connections with employer and title).

**Pass if:**

- The arithmetic adds up and every rejected row has a reason.
- No real person is wrongly merged or dropped.
- Bad emails are flagged but the person is kept; `info@` / no-reply addresses are flagged.
- The do-not-contact address stays suppressed after re-import.

**Write down:** Rows read / valid / duplicate / invalid; any wrong duplicate or wrong company merge.

### *C4 Ranking sanity*

**Goal:** The people ranked highest are genuinely the best people to ask, and every reason is true.

**Have ready:** C3 done. Your target companies and roles.

**Steps:**

1. **Settings** -> set your target roles, target companies and any companies to exclude. Save.
2. Menu **Prospects** -> press **Recalculate** and wait.
3. Open **Why?** on the top 10, and on 5 people picked at random from the middle of the list.
4. Change one weight in **Settings** (e.g. raise company match), press **Recalculate** again.

**Pass if:**

- Every explanation quotes a real fact (title, company, connection date) and claims nothing you cannot verify.
- Nobody is high who obviously should not be (irrelevant recruiter, someone who left long ago).
- Confidence is low where employment information is old or unknown.
- Excluded companies disappear; the order changes in the direction of the weight you changed.

**Write down:** Anyone in the top 20 who should not be there, and why.

### *C5 Jobs and verification evidence*

**Goal:** A job marked open really is open, and a closed one is not called open.

**Have ready:** Real mode. 3-5 companies you care about; one job you **know is closed** and one you **know is open** on the company's own careers site.

**Steps:**

1. Menu **Jobs** -> add the companies' job boards (or paste a job link) and run discovery.
2. Read the *coverage of the last run*: which boards answered, which companies have no reachable board.
3. Open the job you know is open and the one you know is closed. Read **Why this status** and the verification evidence table.
4. Open the referral-route panel of one job.

**Pass if:**

- The open job is `ACTIVE_VERIFIED` only because the company's own board lists it - not just because a page loaded.
- The closed job is not shown as active.
- The referral route says *Unknown* until you point it at an official careers/referral page.

**Write down:** Any job whose status disagrees with the company's site.

### *C6 Resume tailoring and the claim ledger*

**Goal:** A tailored resume contains only things you actually did.

**Have ready:** C1 done (profile corrected and confirmed). A real job from C5 you would apply to.

**Steps:**

1. Open the job (or **Opportunities**) and start tailoring a resume for it. Open the result under **Resumes**.
2. Read **Claim check**: every statement should be marked supported.
3. Download the **PDF** and read it line by line against your own resume.
4. Read *Requirements your background does not support yet* and *Questions for you*.
5. Guard-rail test: edit a bullet to add a technology you have never used. Try to approve. Then undo the edit.
6. Approve the resume, then change something and look at the versions.
7. Open the PDF and DOCX in your normal viewer: text selectable, links clickable, one column.

**Pass if:**

- Nothing on the PDF is something you did not do; no skill appears just because the job asks for it; every number is one you gave.
- The fake technology is flagged and approval is blocked.
- After approval, a change creates a new version and the approved one stays unchanged.

**Write down:** Any sentence on the PDF that is not true or is exaggerated.

### *C7 Outreach drafts (still no sending)*

**Goal:** Drafts are honest and the safety checks block bad messages.

**Have ready:** C3-C6 done. Delivery mode must be **Manual** or **Dry run**.

**Steps:**

1. **Outreach -> Settings**: enter your name and email, acknowledge the lawful-use statement. Confirm delivery mode is Manual.
2. **Outreach** -> **Prepare a new batch**.
3. Read every draft as if you were the recipient. Also read *Things to know* and *What this message claims about you*.

4. Guard-rail tests on one draft, one at a time, trying to approve after each: (a) put `Re:` in the subject; (b) add "as we discussed"; (c) add a technology you do not have; (d) delete the opt-out line. Undo each one.
5. Edit an already approved message and check its approval state.
6. Approve several messages; then press **Pause all**.

**Pass if:**

- No invented relationship, no promised interview, no referral programme claimed unless the company page said so, an easy way to say no, the real job title and official link, an opt-out line in every email.
- Each of (a)-(d) is blocked with a stated reason.
- Editing an approved message withdraws its approval.
- Approving several asks you to confirm the count and your daily limit; when paused nothing can go out.

**Write down:** Any draft sentence that is untrue, pushy, or claims a relationship you don't have.

### *C8 Sending (only when C1-C7 all pass)*

**Goal:** Real sending works, exactly once, only after approval.

**Have ready:** A Google Cloud project with an OAuth client: console.cloud.google.com -> APIs & Services -> enable the **Gmail API** -> OAuth consent screen (External, add yourself as a test user) -> Credentials -> *Create OAuth client ID* -> Web application -> Authorised redirect URI `http://localhost:3000/settings/gmail/callback`. Then create `backend\.env` containing only `GOOGLE_CLIENT_ID=...`, `GOOGLE_CLIENT_SECRET=...` and `ADAPTER_MODE=real`, and restart Terminals 1 and 2.

**Steps:**

1. **Dry run first:** Outreach -> Settings -> delivery mode **Dry run**. Approve one message, press **Send approved emails now**.
2. **Connect:** Outreach -> Settings -> Gmail connection -> tick the consent box -> **Connect with Google** -> sign in and allow. You return to a page saying *Connected as ...*.
3. **Self-test:** press **Send one test email to myself**. Check your inbox and your Gmail Sent folder.
4. **One real message:** switch delivery to Gmail, approve one message to a contact you are comfortable with, send.
5. Open **Tracking**. Then try to send the same message again.
6. On the sent message's page, paste a reply into **Record a reply**; on another, paste a bounce notice. Check what happens.

**Pass if:**

- Dry run reports success and delivers nothing.
- The self-test arrives and is in Sent.
- The real message arrives once; Tracking shows it only after it was really sent; the repeat send is blocked.
- Reply/bounce are classified as described in `docs/COLD_EMAIL_COMPLIANCE.md`; a reply cancels follow-ups; an opt-out is only a suggestion until you confirm it.

**Write down:** Whether each step worked; any error message exactly as shown. Gmail is the least-tested part (simulated until now).

### *C9 Privacy and deletion*

**Goal:** You can take all your data out and remove it completely.

**Have ready:** Any time after C3.

**Steps:**

1. **Settings -> Privacy**: export your data. Open the JSON and search it for *password*, *token*, *key*.

2. Set a short retention period and press **Purge now**.
3. Optional: sign up a second throwaway account in a private window and look for any of the first account's data.
4. With a throwaway account (not your real one): **Settings -> Delete account**, then try to sign in.

**Pass if:**

- The export contains your data and no password hashes, tokens or keys.
- After purge the raw CSV files are gone but your contacts remain.
- The second account sees nothing of the first.
- After deletion, sign-in fails and nothing of that account remains.

## Part D - docker compose (optional, not yet verified)

Only if you want to run it the way a server would. Docker Desktop must be running.

```
cd "C:\Users\anime\Desktop\Referral Portal"
copy .env.example .env      # then edit: set APP_KEK and a real database password
docker compose up --build
```

Generate APP_KEK with: `python -c "import os,base64;print(base64.b64encode(os.urandom(32)).decode())"`

**Pass if:** the four services (db, api, worker, web) start, http://localhost:3000 loads, and you can sign up. Stop with Ctrl+C, then `docker compose down`. This has never been run, so problems here are expected and worth reporting.

## Part E - Results sheet

| Test | Pass / Fail | Notes (numbers, what was wrong) |
|---|---|---|
| B1 Backend SQLite | | |
| B2 Browser | | |
| B3 Backend Postgres (opt.) | | |
| C1 Resume accuracy | | error rate: ___ / ___ entries |
| C2 GitHub / portfolio | | |
| C3 Imports | | read ___ = valid ___ + dup ___ + invalid ___ |
| C4 Ranking | | |
| C5 Job verification | | |
| C6 Tailoring | | |
| C7 Drafts + guard rails | | |
| C8 Sending | | |
| C9 Privacy / deletion | | |
| D docker compose (opt.) | | |

Send back: this sheet, plus a screenshot or redacted example of every wrong extraction, ranking, status or claim. Real data that exposes problems can be fixed with re-import, recalculation and re-versioning - nothing is lost.

## Part F - Troubleshooting

| Symptom | Fix |
|---|---|
| Upload/import stays "queued" | The worker (Terminal 2) is not running. Start it. |
| Web page shows network errors | The API (Terminal 1) is not running on port 8000. |
| Can't find the verification link | Newest file in backend\data\outbox\ - open it in a text editor. |
| "database is locked" | SQLite is single-user. Close other copies of the app; for several users use PostgreSQL. |
| Port already in use | Another copy is running. Close it, or find it: netstat -ano \| findstr :8000 |
| Connect with Google fails | Check the redirect URI is exactly http://localhost:3000/settings/gmail/callback and you are a test user on the consent screen. |
| Job board gives no jobs | You are in mock mode - restart Terminals 1 and 2 with ADAPTER_MODE=real. |
| Start over with an empty app | Stop everything and delete backend\data\ (this deletes all local data). |